

dismissal_evidence_20260826

BOB-206 — dismissal evidence: the uid-flattening claim, refuted by measurement

Revision: 1 Last modified: 2026-08-26T19:40:00Z

Verdict

DISMISSED as written (§11.4.90 reason not - reproducible). The item’s central factual assertion is false on this host, and its stated consequence is inverted from what the mechanism actually produces.

1. The title’s factual claim is refuted

BOB-206’s title asserted “and the download root **IS** such a mount”.
Measured:

declared locations	all six resolve onto one mount
mount	/run/media/milosvasic/DATA4TB
fstype	btrfs
options (verbatim)	rw,nosuid,nodev,relatime,ssd,discard=async,space_cache=v2,subvolid=5,
uid= option	absent
per-inode uid/gid	stored — fully ownership-expressive

Scope was taken from the real resolver (`ownership_scope_entries()`, which expands `${QBITTORRENT_DATA_DIR}` against `.env`), not from assuming `config/owned_paths.yaml` is the only input. The download root is therefore `/run/media/milosvasic/DATA4TB/Downloads` — **not** the `/mnt/DATA` default.

The only uid-flattening filesystem on this host is `/boot/efi` (`vfat`, `fmask=0000`, `dmask=0000`), which is not declared and never will be.

2. The false claim was introduced when the item was written, not by its source

`docs/qa/T041/review_scope_map_20260826.md:296-299` says the download root is “a host-specific mount **whose filesystem the reviewer must not assume.**”

The preflight was careful. The item’s author (the conductor) turned that careful hedge into a positive assertion. Recorded here per §11.4.6 rather than quietly corrected: the error is in the transcription step, and the transcription step is where the same class of error has now occurred more than once this session.

3. The mechanism reproduces — the consequence inverts

`probe_location()` (`scripts/lib/ownership.sh:299-320`) reads `stat -c '%u'` and compares against `id -u`. **It has no filesystem-expressiveness guard** — that half of the hypothesis is CONFIRMED and needle-proven (`findmnt 0`, `mountinfo 0`, `stat -f 0`, `statfs 0`, `fstype 0`, `vfat|exfat|ntfs|msdos 0`, `df -T 0`, `f_type 0` across all four ownership sources; needle `probe_location = 7` hits through the identical instrument; negative control 0. The 57 raw `uid=` hits are **carriers** — 4 are `uid=` assignments, the rest `PUID=` prose; zero are mount-option `uid=`).

Driven privilege-free by shimming `stat` on `PATH` (a faithful model of factor 6 at the exact seam, since uid-flattening is “stat reports the mount uid”). Instrument proven seeing before any result was trusted: `FLATTEN_UID=4242 → wrong-owner:4242`; shim inert → ok.

simulated mount	verdict	correct?
<code>vfat,uid=1000</code> (== operator)	ok rc=0	yes

simulated mount	verdict	correct?
vfat,uid=0	wrong-owner:0 rc=1 — refuses	yes
vfat,uid=100999	wrong-owner:100999 rc=1 — refuses	yes

The ok reproduces but **is not a bluff**: it occurs only when the flattened uid equals the operator's, and in exactly that case every file on the mount — including container-written ones — is accessible to the operator. The user-visible goal is met, by the mount instead of by per-file ownership. The item's blast-radius claim ("the operator still has to chown by hand") describes the uid=0 / uid=100999 cases, and those refuse correctly.

Honest limit (§11.4.3): the shim models the READ, not the WRITE. It cannot settle whether chown fails on such a mount. That residual is retained as a separate operator-gated item, not buried in this dismissal.

4. What the investigation found instead

Minted as separate items rather than left inside a dismissed one:

- **BOB-207** (critical) — `probe_location()` is **GID-blind**: repair fixes uid AND gid, the precondition verifies uid only. Live on this host, unprivileged, two commands.
- **BOB-208** (major) — the want side is unchecked; a failing `id(1)` yields a false refusal that names the correct uid as wrong.
- **BOB-209** (major) — the two stat guards are asymmetric halves; a stat failure is mislabelled unwritable; the probe temp file has no EXIT trap (§11.4.14).
- **BOB-210** (major) — `detect_rootless()`'s header promises an unverified reading can never manufacture a refusal, but `:413` emits a confident `rootful` that reaches the R3 refusal; and the shim oracle shares the code's unvalidated premise (§11.4.245).
- **BOB-211** (minor, latent + operator-gated) — vfat repair-diagnosis and `fmask/dmask` mode-contract residuals; RED needs `bindfs` (absent) or a privileged loopback.

5. Corrections owed to the T041 preflight record

The preflight was **right** that factor 6 is unguarded and untested, **right** about R4's three terms, **right** that the docker branch is unmeasured, and **careful** not to claim the download root's fstype.

It was **wrong** on one point: R4 does **not** lack a second line of defence. The FR-011 gate's Invariant 4 (scripts/pre_build/check_cm_ownership_invariants.sh:658-888) does read the compose user: key, and additionally reads the Dockerfile USER directive (:855). The source states two layers exist (scripts/ownership_precondition.sh:761-765). The preflight quoted the **historical justification** for R4's creation (finding IMPORTANT-1, 2026-08-21) as if it were the current state.

6. Also verified good — recorded so it is not re-investigated

- The unknown: branch is handled **identically** at both consumers (R3 :519-531, R4 :832-836 — both a named SKIP, both return 0, neither refuses).
- ROOTLESS_VERDICT is cached at :487, so both checks consume **one** measurement — the source's own warning that the two must never refuse on different facts is honoured.
- The carrier-trap golden-FALSE fixture **exists** (tests/unit/test_ownership_rootless_detection.sh:279-308 drives name=rootless-lookalike inside a profile path and must NOT read rootless; structural guard at :396-411 uses exact field equality, not substring).
- **No test anywhere** exercises a uid-flattening filesystem — vfat/exfat/ntfs/flatten/ loopback/mkfs/losetup/bindfs/fusermount all 0 across the three ownership test files (needle probe_location = 3 in the same files).

Constraints honoured

Nothing mounted, no sudo, no `pre_build_verification.sh`, no commits or stages, no edits to `docs/Issues.md` / `docs/Fixed.md` / `docs/workable_items.db` by the verifying agent, and it stayed off the sibling stream's `lan_route_auth_analyzer.py`. Zero probe residue left (needle-proven the find could see).